

DevOps, DevEx & Platform Engineering

Des origines Agile à la DX

Formateur : Alex FAIVRE

Année : 2025-2026



| Plan du Cours (1/2)

- ✓ **1. Le Contexte & L'Histoire** : Comprendre l'origine du mouvement.
- ✓ **2. Les Fondations du DevOps** : Le modèle C.A.L.M.S. en détail.
- ✓ **3. Agile vs DevOps** : Continuité et différences.
- ✓ **4. Les "Three Ways"** : Les principes de flux, feedback et apprentissage.

| Plan du Cours (2/2)

- ✓ **5. La Transition** : Charge Cognitive et complexité.
- ✓ **6. Developer Experience (DevEx)** : Flow, Feedback et Facteur Humain.
- ✓ **7. Platform Engineering** : L'industrialisation au service du DevEx.
- ✓ **8. Mesurer la Performance** : Les métriques DORA.



Partie 1 : Contexte Historique

Pourquoi le logiciel allait-il dans le mur ?

DevOps - Une brève histoire de temps

2007 - Patrick Debois

La conf Velocity 2009 par Flickr (Allspaw & Hammond)
prouve que Dev et Ops peuvent coopérer.

10+ Deploys per Day

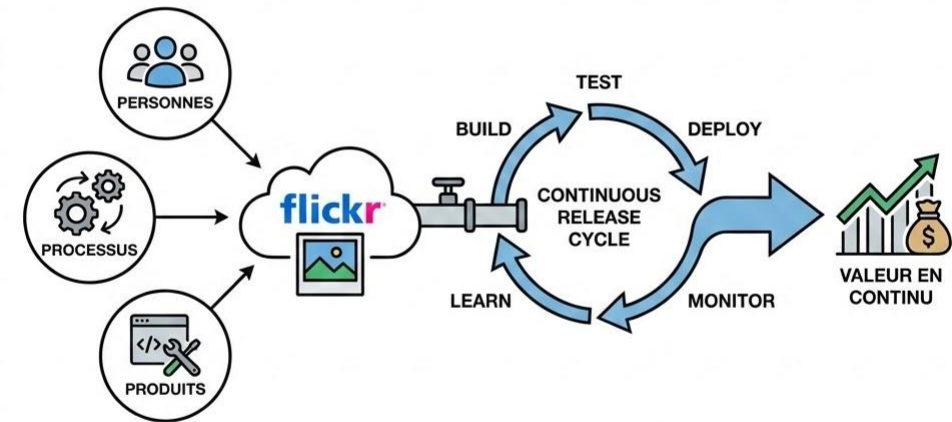
La conf Velocity 2009 par Flickr (Allspaw & Hammond)
prouve que Dev et Ops peuvent coopérer.

DevOps = Culture + Process + Tools

C'est l'union des personnes, des processus et des
produits pour fournir de la valeur en continu.

10+ Deploys per Day

La conf Velocity 2009 par Flickr (Allspaw & Hammond)
prouve que Dev et Ops peuvent coopérer.



C'est l'union des personnes, des processus et des
produits pour fournir de la valeur en continu.

Le problème - Le "Mur de la Confusion"

Conflit d'intérêts

D'un côté, les **Devs** veulent aller vite (Change).

De l'autre, les **Ops** veulent éviter les pannes (Stability).

Ce mur crée des silos hermétiques où le code est "jeté" à l'aveugle, générant frustration et dette technique.

L'incompréhension mutuelle étant le moteur de frustration et paralysie le plus fort pour ruiner les efforts de chacun.

Le Point de Départ : Le "Mur de la Confusion"





Partie 2 : Le Modèle C.A.L.M.S.

Culture, Automation, Lean, Measurement,
Sharing

C - Culture & A - Automation

Culture (Humain)

Responsabilité Partagée : "You build it, you run it".

Blameless Post-Mortems : L'erreur est une opportunité d'apprentissage, pas de punition.

Automation (Technique)

Éliminer le Toil : Automatiser les tâches manuelles répétitives.

Infrastructure as Code (IaC) : Traiter les serveurs comme du logiciel (Git, Versionning, Tests).

DEVOPS: CULTURE & AUTOMATISATION

AMÉLIORATION CONTINUE & LIVRAISON RAPIDE

CULTURE DEVOPS



CULTURE DEVOPS

COLLABORATION
COMMUNICATION
COMMUNICATION
RESPONSABILITÉ PARTAGÉE

AUTOMATISATION DEVOPS



AUTOMATISATION DEVOPS

PIPELINE CI/CD
EFFICACITÉ
TESTS AUTOMATISÉS
DÉPLOIEMENT RAPIDE

L - Lean

Optimiser le Flux

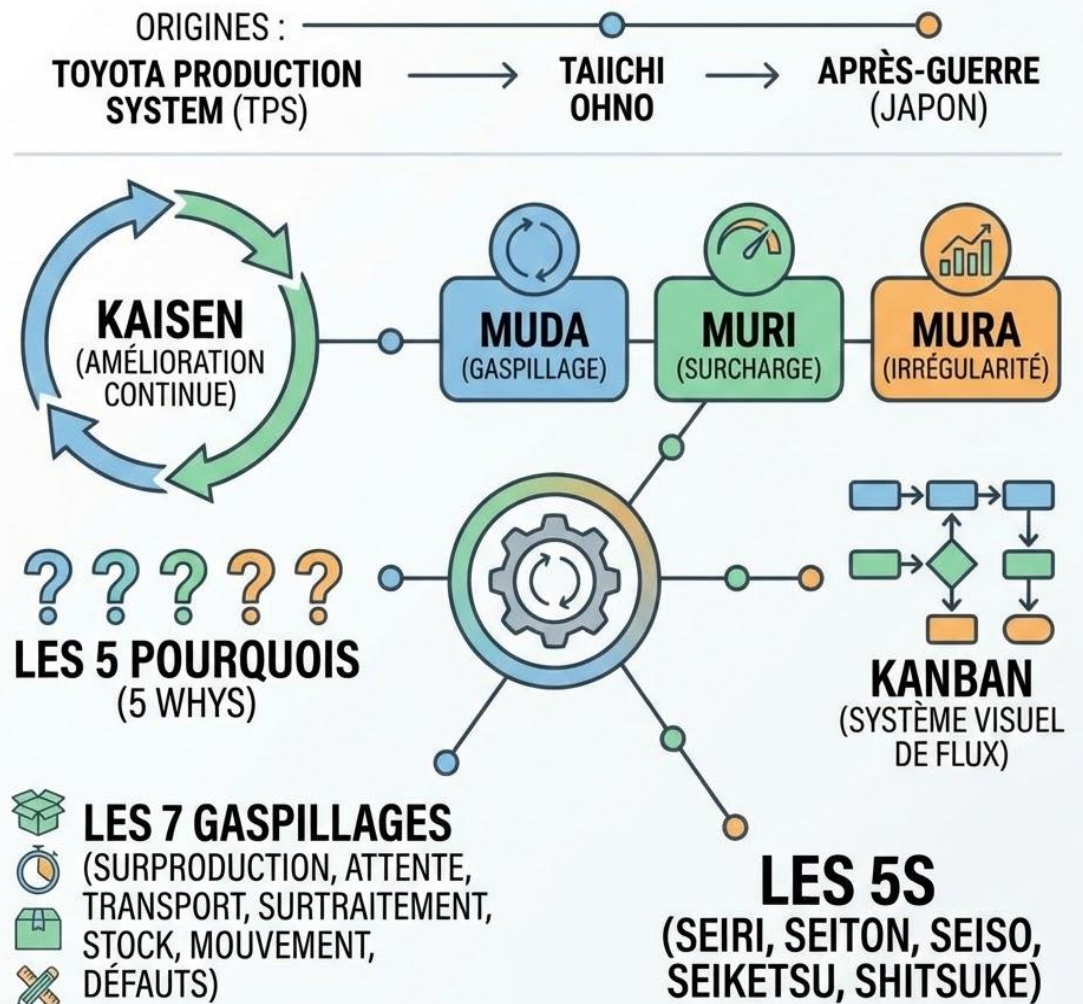
Inspiré du Toyota Production System.

- **Small Batches** : Déployer petit réduit le risque et facilite le rollback.
- **Value Stream Mapping** : Visualiser toutes les étapes pour identifier les goulots d'étranglement.

5S : Cinq étapes Japonaises

- Seiri (Trier)
- Seiton (Ranger)
- Seiso (Nettoyer)
- Seiketsu (Standardiser)
- Shitsuke (Maintenir / Discipliner)

LEAN : ORIGINES & PRINCIPES CLÉS



M - Measurement & S - Sharing



Measurement

Mesurer pour améliorer (Métriques Tech & Business). La donnée remplace l'opinion.
Notion de Data Driven.



Sharing

Briser les silos d'information. Guildes, Documentation ouverte, retours d'expérience.



Partie 3 : DevOps & Agile

Comprendre la filiation

| L'extension naturelle de l'Agile

De l'idée à la production

Agile a connecté le Business et le Développement.

DevOps connecte le Développement et les Opérations.

Objectif commun : Réduire le temps entre une idée et sa valeur livrée à l'utilisateur.



Partie 4 : Les "Three Ways"

Les principes de "The Phoenix Project"

Les 3 Voies de Gene Kim



1. Flow

(Dev -> Ops)

Optimiser la vitesse du système global. Éliminer les blocages.



2. Feedback

(Ops -> Dev)

Boucles de rétroaction rapides.
Monitoring, logs, alertes immédiates.



3. Learning

(Continu)

Expérimentation, Chaos Engineering,
Répétition.



Partie 5 : La Crise de la Complexité

Quand le "You Build It, You Run It" devient
insupportable

L'explosion de la Charge Cognitive

Le fardeau du Full Stack

Aujourd'hui, un développeur doit connaître : Java/Python, mais aussi Docker, K8s, Helm, Terraform, IAM, SecOps, Observability...

Cette charge excessive tue la productivité et l'innovation.



**NOS DÉVELOPPEURS
EN OPS AMATEURS.
ILS PASSENT PLUS DE
TEMPS À CONFIGURER
DU YAML QU'À CODER
DES FEATURES.**



Partie 6 : Developer Experience (DevEx)

Remettre l'humain au centre

Qu'est-ce que le DevEx ?

Définition

La **DevEx** se concentre sur la manière dont les développeurs perçoivent leur travail.

Ce n'est pas (seulement) une question d'outils, mais de **friction**.

**Bonne DX =
Moins de frictions &
+ d'impact**



Les 3 Dimensions du DevEx

Selon le framework de Nicole Forsgren (GitHub/Microsoft)



Flow State

La capacité à se concentrer profondément sans interruptions. C'est là que la productivité et la joie résident.



Feedback Loops

La vitesse et la qualité des réponses du système (CI/CD, Tests). Attendre 20min un build tue le Flow.



Cognitive Load

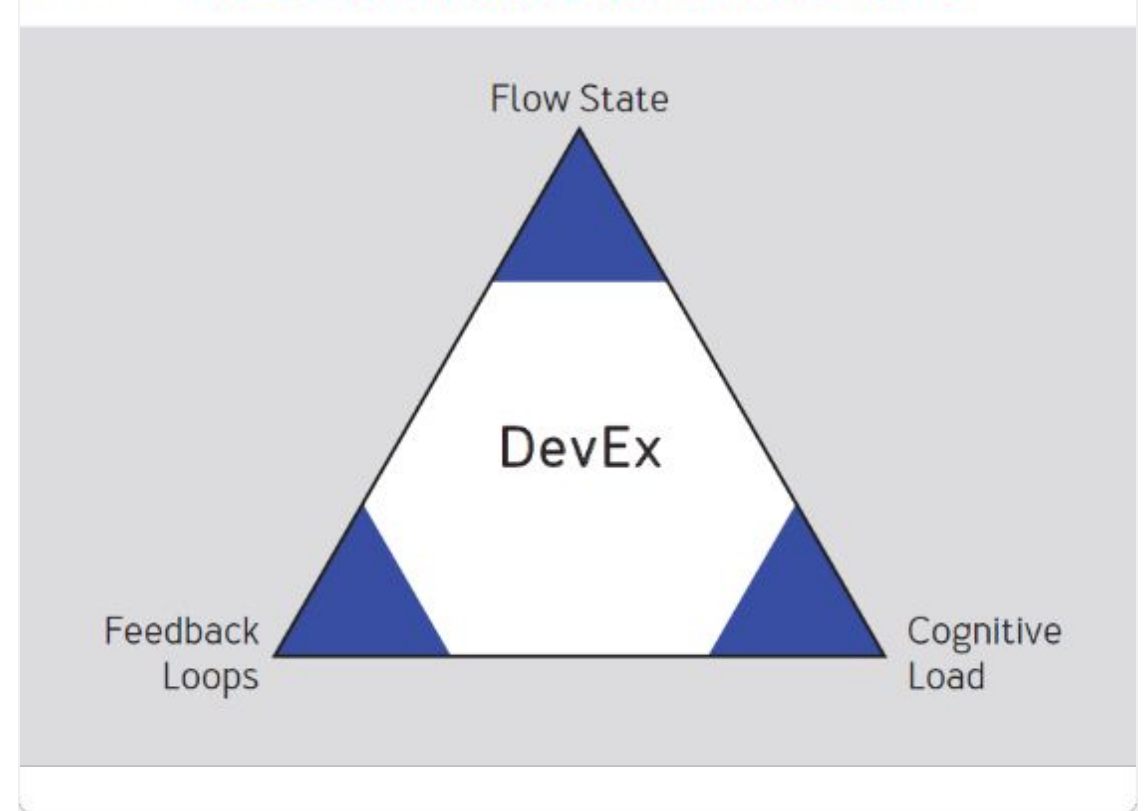
La quantité d'effort mental nécessaire pour accomplir une tâche. Une documentation pauvre augmente cette charge.

Le Framework DevEx

Pourquoi c'est important ?

- ✓ **Productivité** : Un développeur en "Flow" produit un code de meilleure qualité, plus vite.
- ✓ **Rétention** : Les développeurs frustrés par des outils lents ou complexes démissionnent.
- ✓ **Sécurité** : Un système simple à utiliser est un système plus sûr (moins d'erreurs humaines).

FIGURE 1: **THREE CORE DIMENSIONS OF DEVELOPER EXPERIENCE**





Partie 7 : Platform Engineering

La réponse technique au problème de DevEx

Team Topologies : La Structure

Comment organiser les équipes pour maximiser le DevEx ?

Stream-aligned Teams

Les équipes produits (Devs). Elles doivent être concentrées sur le flux de valeur business (Flow).

Platform Teams

Elles construisent la "route" pour les Stream-aligned teams. Leur but : Réduire la charge cognitive des Devs.

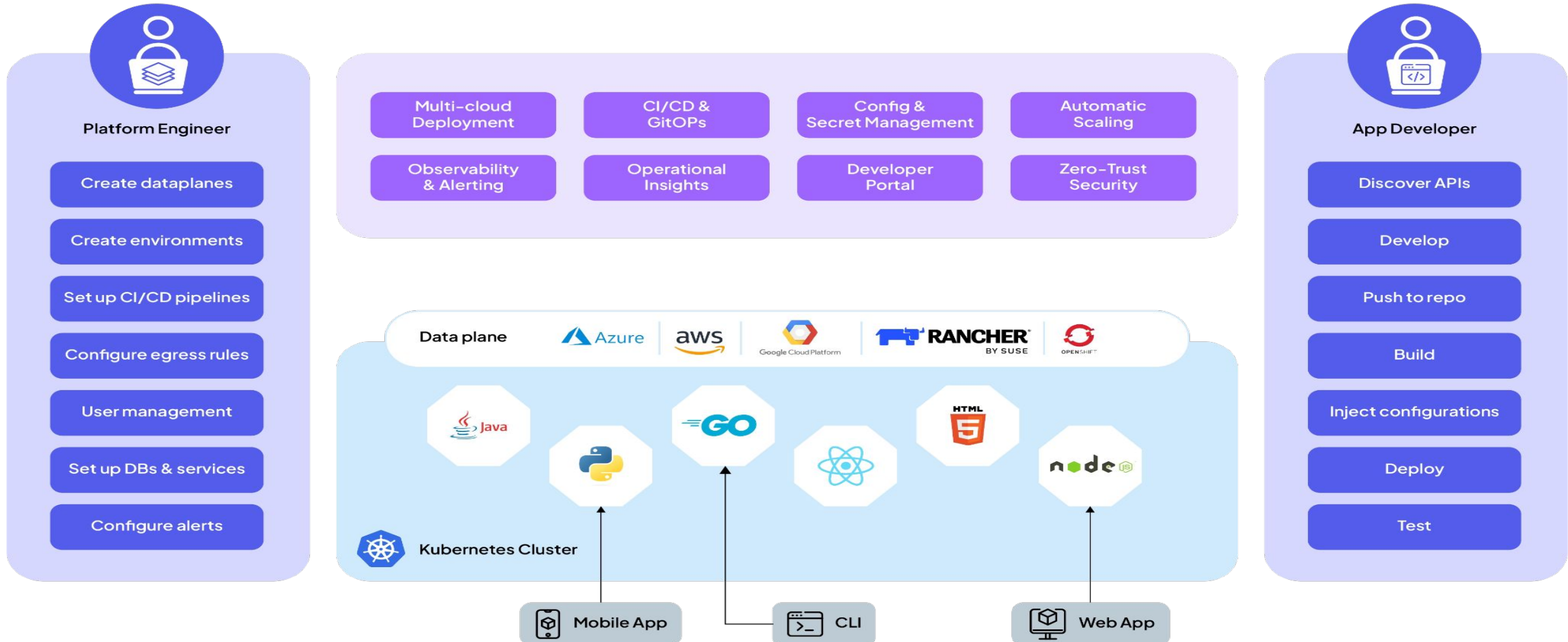
L'IDP (Internal Developer Platform)

L'outil principal du Platform Engineering.

✓ **Abstraction** : Masque la complexité de Kubernetes/Cloud.

✓ **Self-Service** : Le développeur est autonome.

✓ **Golden Paths** : Des modèles standards et sécurisés par défaut.



| Platform as a Product

| *"Traitez votre plateforme comme un produit et vos développeurs comme des clients."*

— *Principe fondamental*

Si la plateforme est plus difficile à utiliser que AWS en direct, personne ne l'utilisera.



Partie 8 : Mesurer le Succès

DORA

Metrics

Les Métriques Clés (DORA METRICS)

Standard de l'industrie (Google).

- **Deployment Frequency** (Vitesse)
- **Lead Time for Changes** (Vitesse)
- **Change Failure Rate** (Stabilité)
- **Time to Restore Service** (Stabilité)

DORA METRICS

A/F



DEPLOYMENT FREQUENCY

How often code is deployed.
High performers deploy on demand.



LEAD TIME FOR CHANGES

Time from code commit to production.
Lower is better.



CHANGE FAILURE RATE

Percentage of deployments causing failure.
Lower is better.



TIME TO RESTORE SERVICE

Time to recover from incident.
Faster is better.



| Conclusion

- ✓ Le **DevOps** a brisé les silos (Culture).
- ✓ Le **Platform Engineering** gère la complexité technique (Outils/Process).
- ✓ Le **DevEx** s'assure que l'humain reste efficace et heureux (Flow).
- ✓ Les trois sont indissociables pour une IT performante.

Fin du cours

Questions

?

 Pour aller plus loin :

"DevEx: What Actually Drives Productivity" (Forsgren et al.)

"Team Topologies" (Skelton & Pais)